

An Implementation on Ransomware Detection Using Deep Learning

Pradip Ghimire

Final Year MCA Student, Dept of MCA, School of CS & IT, Jain Deemed-to-be University,
Bengaluru,

Dr. A. Rengarajan,

Professor, Dept of MCA, School of CS & IT, Jain Deemed-to-be University, Bengaluru.

ABSTRACT

Ransomware is a malicious software that has been on the rise in recent years. It often encrypts the user's files or systems and retains the decryption key until the victim pays a ransom, which is usually in bitcoins due to their untraceability. The purpose of this study is to present a novel deep learning-based technique for detecting ransomware targeting Microsoft Windows that accepts files in standard format as input and returns a prediction of whether the file is benign, generic malware, or ransomware. The model's ability to distinguish between ransomware and other forms of malware allows it to be applied as an extension to an existing malware detection system such as anti-virus software, and aid in the categorization and reverse engineering of new in-the-wild ransomware samples. The model's capacity to tell the difference between ransomware and other types of malware. The ensemble automates the static analysis of Windows software binaries by extracting features from the files' contents and abstracting patterns within those features. The model's predictive potential in identifying new in-the-wild samples is expected to be strong as the model is tested on data.

Keywords: threat, bitcoin, benign, malware, software binaries

1. INTRODUCTION

The first step in detecting ransomware is to use automation and malware analysis to identify potentially harmful files early in the kill chain. The cyber kill chain is a cybersecurity concept that monitors the stages of a cyber-attack, identifies vulnerabilities, and assists security teams in stopping attacks at each level. In most cases, the detection and analysis procedure includes compiling a list of potentially suspicious behaviours in order to determine whether or not something malicious is taking place. To properly combat today's ransomware strains, new technologies and methods are required. Traditional detection and security methods, which rely on signature-based detection or abnormal traffic monitoring, are ineffective against modern ransomware threats. Using technologies that identify anomalous file behaviour is an efficient way to detect and prevent ransomware activity as soon as possible. Deep learning detection is an effective method for detecting new strain of ransomware.

Deep Learning is used to create behavioural analytics systems that have been trained to detect unusual file behaviour. These systems are an excellent way to detect and prevent ransomware infections from spreading through the file system. Solutions that use deep learning can detect anomalies in file behaviour, which may include changes made by ransomware.

This project is purely based on predicting the malicious files by trained module. The module was trained by using a large dataset. Extracted binaries files are used for the dataset. The binary files used Microsoft's standard Portable Executable format for executable files on Windows, found in files with '.exe' and '.dll' extensions. A compiled Windows executable is distributed for a terminal program which allows the end-user to specify the path of a PE file and receive the model's classification of the file as either benign, malware, or ransomware.

This proposed system provides a simple interface to the model that should allow users without any expert knowledge to detect if a portable executable file is likely to be ransomware. This requirement was elicited from the fact that a great variety of individuals and organizations are affected by ransomware attacks, and so a good detection system should be simple and accessible for any individual. The experiment conducted for this project is purely lab-based, using almost exclusively objective numerical data, and aimed to test predictions made from null and alternative hypotheses.

2. Related work

This research attempted to detect ransomware using an ensemble of ML (Machine Learning) classifiers, categorizing Windows executable files as benign, malicious, or ransomware. The hope is that by making the model's source code available to the public, it can be used in practice to identify ransomware on a user's machine, or it can be integrated into an existing AV (Anti-Virus) system to help or improve its ability to distinguish between ransomware and other types of malware.

The impact of ransomware attacks peaked in 2017 (Beazley[16]), with the infamous WannaCry worm spreading across the globe, disabling the essential digital services of organizations ranging from FedEx to NHS England, and demanding a ransom paid in the relatively anonymous cryptocurrency Bitcoin (Mohurle and Patil[17]).

With the number of new ransomware variants growing (Symantec[18]), detection techniques must keep up. Traditionally, malware detection systems such as commercial AV software relied on a signature-based approach, recording identifying details in a central database, and updating a copy stored on the user's computer.

Their most effective technique was an ensemble of Naive Bayes classifiers trained on the raw bytes of executable files, which produced a malware detection rate of 97.76 percent, approximately three times higher than the signature method, which was evaluated at only 33.75 percent (Schultz et al.[12]). Again, the 3622 sample files utilized were likely insufficient to accurately generalize these results to the entire population of in-the-wild malware, which was believed to be over 50,000 executables at the time (Microsoft[19]).

Vinayakumar et al.[20]discovered that a sort of DLNN (Deep Learning Neural Network) called an MLP (Multi-Layer Perceptron) performed better than an SVM in both multi-class (98%) and binary (100%) classification of executable files as benign or belonging to one of seven different ransomware families. This study, however, like many recent investigations into ML for ransomware detection, used a small sample, just 974 executable files (Vinayakumar et al.). Considering that NTT Security's Global Threat Intelligence Report notes that ransomware comprised 7% of all malware (NTT Security), and a reported total of 856.62M malware files that same year, there should be approximately 59.9M ransomware files in-the-wild as of 2018. For a confidence level of 95% and a confidence interval of 1%, a representative sample of the in-the-wild population of ransomware should contain at least 9,603 files.

CF-NCF (Class Frequency - Non-Class Frequency) was proposed by Seong II Bae et al,[1] feature vectors were created using CF-NCF, and six machine learning techniques were tested to detect ransomware. CF-NCF is a classification model indicator that is based on TFIDF (Term Frequency - Inverse Document Frequency). The suggested ransomware detection approach can distinguish ransomware from both benign and malicious files. It was published in 2019 and used 300 benign files, 1000 ransomware samples, and 900 malware samples as sample input files. The proposed method can detect ransomware n files with an accuracy of up to 98.65%.

As well as processing malware such as ransomware as text, researchers have achieved accurate classifications when representing features as images. Nataraj et al.[22] originally proposed representing the raw bytes of PE files as 8-bit greyscale images and used them as input for a machine learning classifier in ‘Malware Images: Visualization and Automatic Classification, using the kNN (k- Nearest Neighbours) clustering algorithm (Nataraj et al.,). This was replicated in a 2018 study with much larger sample size and a deep 2-D CNN achieving 99.97% accuracy in multi-class Classification (Kalash et al.[23]) on the popular and sizable benchmark Microsoft Malware Classification Challenge dataset (Ronen et al., 2018). CNN's achieve state-of-the-art accuracy in classifying images (Gu et al.[24]), which can also be generated from relatively long sequences of data that other models may fail to learn from (Singh et al.[25]).

3. System Architecture

Python is a sensible choice of PL (Programming Language) for an ML project, with Kaggle’s most recent Machine Learning and Data Science Survey finding it far and away from the most popular PL in the DS (Data Science) and ML industry, as well as the most frequently recommended as a first language to learn ML and DS. Being high-level, interpreted, and dynamically typed, and with support for OOP (Object Oriented Programming), Python lends itself to the constant pace and re-use of code between versions associated with the iterative development model.

The PC was equipped with relatively powerful hardware: 12GB of RAM for simultaneous pre-processing of large samples parallelized on a 4GHz octa-core AMD FX-8350 CPU using the Joblib Python package, a 500GB hard drive to store the large number of samples required for accurate classification and their even larger ML ready representations, and an Nvidia GeForce GTX 770 GPU with 2GB of memory and 1536 CUDA cores for TensorFlow to use for GPGPU (General Purpose Computing on GPU), parallelizing the training of DL models and in turn massively decreasing the time required for training.

The project used free and open-source python packages like TensorFlow to allow for rapid development and prototyping. Capstone disassembler Python API in combination with the file

Python package to disassemble PE files into assembly code and extract the opcodes. Other Python packages used include the aforementioned Joblib for parallel execution of Python functions, as well as NumPy for efficient representation and manipulation of multi-dimensional arrays, scikit-learn which provides useful tools for ML such as splitting datasets into the train, validation, and test sets, h5py for efficient non-volatile representation of ML model weights, Matplotlib and Seaborn for visualization of data and model performance, alongside pretty-print-confusion-matrix, which provides a simple function for generating a confusion matrix.

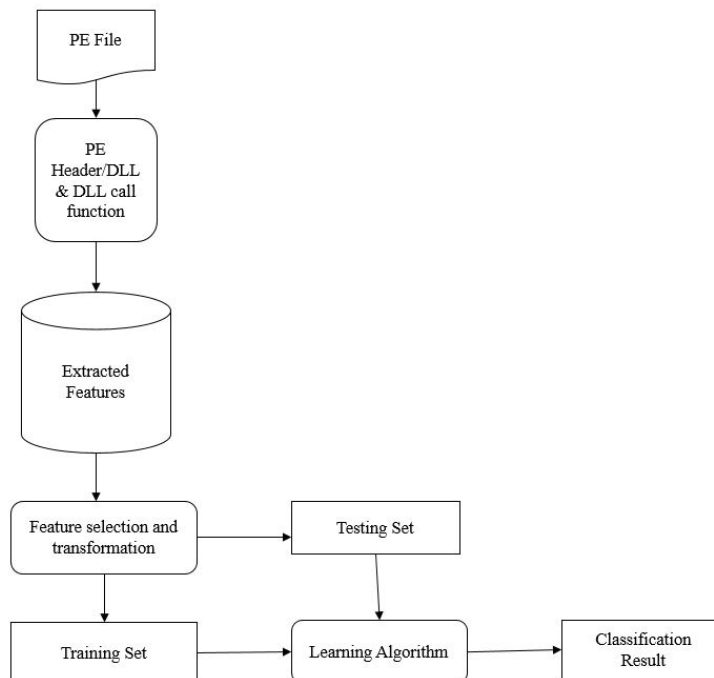


Fig. 1 Proposed System Architecture

The Portable Executable (PE) format is a file format used in Windows 32-bit and 64-bit versions for executable, code, DLLs, FON Font files, and so on. The PE format stores the knowledge needed by the Windows OS loader to manage the wrapped executable code. The PE header explains the Microsoft PE executable format, with the image header's base at the top. Deep learning deciphers higher-level data at alternative layers supporting raw input file by using layers of neural network algorithms. Neural networks will help cluster points at intervals in a large sample of knowledge based on the commonalities of its options, classify knowledge based on labels from earlier knowledge, and extract different alternatives from knowledge. The numerical patterns that these networks recognize are stored in vectors that represent real-world inputs. Deep neural networks can be viewed as components of broader machine-learning systems that include algorithms for reinforcement learning, classification, and regression. Deep learning employs self-taught learning and algorithmic program constructions with multiple hidden layers, as well as large amounts of data and sophisticated computer resources.

The deciliter ensemble model built as part of this study automates static analysis and categorization of Windows feasible files as benign, malware, or ransomware with excellent accuracy.

4. Implementation

Two ML models were developed from different feature sets extracted from a shared set of binary files. The binary files used Microsoft's standard PE format for executable files on Windows, found in files with '.exe' and '.dll' extensions.

To train, test, and evaluate the project's ML models, around 30,000 sample PE files were gathered, roughly 10,000 for each class, so as not to bias the model toward any particular classification. The benign files were downloaded from web links scraped from the HTML code of 'freewarefiles.com' using the Python web-scraping library BeautifulSoup. The malware samples were obtained via torrents of the latest generic malware detections available from the free and public online malware repository 'virusshare.com', which offer more than 45M assorted malware samples total, separated into zip archives categorized by date and time. Several archives were extracted and PE file samples were detected and moved into a directory labeled 'malware samples'.

The first model was trained using a numerical vector representation of words retrieved from the PE files' raw bytes. Words were created by decoding every byte of a file as a utf-8 encoded character, independent of intended encoding, then combining characters to make words, delimiting words with whitespace characters like spaces and tabs.

The NLP (Natural Language Processing) technique of feature hashing was used to numeracies these words. Also known as the 'hashing trick', feature hashing uses a hashing algorithm to generate integers from words, with each unique word input into the function corresponding to a unique integer output. The technique is memory, storage, and time-efficient compared to the common bag-of-words approach for representing text data. It does not require a record of the association between each word in a corpus of text documents and its numerical representation, or the computational and time requirements of searching a large dictionary for a word to find its associated value. These benefits were essential for a feasible representation of the extracted words, as the vocabulary of all the unique words present in the feature set totaled over 11.6M words, which would require searching a dictionary of the same size each time a word is converted to an integer in a typical bag-of-words approach.

The second model was trained on a far smaller number of features: a probability distribution of the appearance of opcodes within the assembly code of a file, inspired by the 97% accuracy of 'Malware Detection Based On Opcode Frequency'. The opcodes were extracted using the open-

source Capstone disassembly engine, and PE file Python APIs and the frequency of opcodes was measured for the entire corpus to find the top-50 most frequent. The frequency of each of these top-50 opcodes was measured for each sample and normalized into the probability of its appearance by dividing it by the total number of opcodes in the file's disassembled assembly code. This produced a vector of 50 probabilities using a bag-of-words approach, which suited this data more so than the utf-8 strings used by the other model, due to the more manageable vocabulary size of 50 opcodes. During training, the accuracy of predictions on both the training data, and a validation set of data unseen by the model was used to test the performance of both strings and opcode classification models, and models were tweaked, redesigned, and redeveloped iteratively to select the model and pre-processing designs which lead to the greatest validation accuracy, the standard process for ML model selection.

5. Analysis

After the architectures with high validation accuracy were designed and implemented, the results of their performance in classifying unseen test data were surprisingly impressive, with 96 percent test accuracy for the ensemble model, considering their ability to compete with contemporary research in the field, which is frequently performed by teams of academics. And when the ensemble model was adapted to perform binary classification of executables as either benign or malicious, the test accuracy increased to 98 percent, which is the most important distinction for end-users of the model who use it to determine whether or not a Windows program is safe to execute.

Though the 96 percent test accuracy of the ensemble model's 3-class predictions is among the highest values achieved in contemporary malware and ransomware detection research, there are multiple studies that surpass it.

6. Conclusion

This project's Deep Learning model automates static analysis and categorization of Windows executable files as benign, malware, or ransomware with high accuracy. The model is shown to properly categorize sample files that were not observed during training, implying that it is generalizable to detecting malware and ransomware in the wild. It is vital to the security of end-users of a simple terminal software built as part of this research for ransomware detection that the differentiation between benign and dangerous files be accurate, and this was tested to be 98 percent accurate on a set of samples.

This performance is comparable to recent studies into ransomware and malware detection. Anti-virus developers and security researchers can use and re-use the ensemble model in their own systems, and retrain the models with a larger dataset and more recent samples to maintain its ability to classify executables in the wild.

REFERENCES

- [1] Seong Il Bae, Gyu Bin Lee, Eul Gyu Im, Ransomware detection using machine learning algorithms, <https://doi.org/10.1002/cpe.5422>, 2019
- [2] S.H. Kok , A. Azween, NZ Jhanjhi , Evaluation metric for crypto-ransomware detection using machine learning, <https://doi.org/10.1016/j.jisa.2020.102646>, 2020
- [3] Greg Cusack, Oliver Michel, Eric Keller, Machine Learning-Based Detection of Ransomware Using SDN, <https://doi.org/10.1145/3180465.3180467>, 2018
- [4] BojanVujančić, NemanjaMaček, SašaAdamović, AN IMPLEMENTATION OF RANSOMWARE MALICIOUS SOFTWARE IN PYTHON, DOI:10.15308/Sinteza-2017-19-24, 2017
- [5] Gavin Hull1 , Henna John2 and Budi Arief, Ransomware deployment methods and analysis, <https://link.springer.com/article/10.1186/s40163-019-0097-9>, 2019
- [6] Group of McAfee Labes, Understanding Ransomware and Strategies to Defeat it, <https://www.mcafee.com/enterprise/en-us/assets/white-papers/wp-understanding-ransomware-strategies-defeat.pdf>, 2017
- [7] Asibi O. Imaji, Ransomware Attacks: Critical Analysis, Threats, and Prevention methods, https://www.researchgate.net/publication/332551447_Ransomware_Attacks_Critical_Analysis_Threats_and_Prevention_methods, 2019
- [8] Sonu B. Surati, Ghanshyam I. Prajapati, A Review on Ransomware Detection & Prevention, <https://www.rsisinternational.org/IJRSI/Issue46/86-91.pdf>, 2017
- [9] Savita Mohurle, Manisha Patil, A brief study of Wannacry Threat: Ransomware Attack 2017, ISSN No. 0976-5697, 2017

- [10] Hong-Yi Chang, Tu-Liang Lin, Implementation of ransomware prediction system, DOI: 10.1109/ICCE-TW46550.2019.8991771, 2017
- [11] FarnoushManavi, Ali Hamzeh; A novel approach for ransomware detection based on PE header using graph embedding; January 2022; Journal of Computer Virology and Hacking Techniques; DOI:10.1007/s11416-021-00414-x
- [12] Schultz, M.G., Eskin, E., Zadok, F. and Stolfo, S.J. (2001) Data mining methods for detection of new malicious executables. In: Proceedings 2001 IEEE Symposium on Security and Privacy. S&P 2001. 2001 Oakland, CA, USA: IEEE Comput. Soc, 38–49.
- [13] Feng, Y.; Liu, C.; Liu, B. Poster: A new approach to detecting ransomware with deception. 22–24 May 2017, https://www.ieee-security.org/TC/SP2017/poster-abstracts/IEEE-SP17_Posters_paper_26.pdf
- [14] Mohammad Mehdi Ahmadian; Hamid Reza Shahriari; Seyed Mohammad Ghaffarian, Connection-monitor & connection-breaker: A novel approach for prevention and detection of high survivable ransomwares. In Proceedings of the 2015 12th International Iranian Society of Cryptology Conference on Information Security and Cryptology (ISCISC), Rasht, Iran, 8–10 September 2015; DOI: 10.1109/ISCISC.2015.7387902
- [15] Almashhadani, A.O.; Kaiiali, M.; Sezer, S.; O’Kane, P. A multi-classifier network-based crypto ransomware detection system: A case study of locky ransomware. IEEE Access 2019, DOI: 10.1109/ACCESS.2019.2907485
- [16] Beazley (2018) 2018 Breach Briefing, whitepaper, <https://www.beazley.com/documents/Whitepapers/201802-beazley-breach-briefing.pdf>
- [17] Mohurle, S. and Patil, M. (2017) A brief study of Wannacry Threat: Ransomware Attack 2017. International Journal of Advanced Research in Computer Science, 4, DOI: <https://doi.org/10.26483/ijarcs.v8i5.4021>
- [18] Symantec (2018) ISTR (Internet Security Threat Report). California, US: Symantec. <https://docs.broadcom.com/doc/istr-23-2018-en>
- [19] Microsoft (2012) The Evolution of Malware and the Threat Landscape - a 10-year Review. Security Intelligence Report. Washington D.C., US: Microsoft.
- [20] Vinayakumar, R., Soman, K.P., Velan, K.K.S. and Ganorkar, S. (2017) Evaluating shallow and deep networks for ransomware detection and classification. In: 2017 International Conference on Advances in Computing, Communications and Informatics (ICACCI). September 2017 Udupi: IEEE, 259–265.
- [21] NTT Security (2018) 2018 Global Threat Intelligence Report.
- [22] Nataraj, L., Karthikeyan, S., Jacob, G. and Manjunath, B.S. (2011) Malware Images: Visualization and Automatic Classification. In: Proceedings of the 8th International Symposium on Visualization for Cyber Security. VizSec ’11. 2011 New York, NY, USA: ACM, 4:1–4:7.
- [23] Kalash, M., Rochan, M., Mohammed, N., Bruce, N.D.B., Wang, Y. and Iqbal, F. (2018) Malware Classification with Deep Convolutional Neural Networks. In: 2018 9th IFIP

International Conference on New Technologies, Mobility and Security (NTMS). February 2018 1–5.

- [24] Gu, J., Wang, Z., Kuen, J., Ma, L., Shahroudy, A., Shuai, B., Liu, T., Wang, X., Wang, L., Wang, G., Cai, J. and Chen, T. (2017) Recent Advances in Convolutional Neural Networks.arXiv:1512.07108 [cs], Available from <http://arxiv.org/abs/1512.07108> [accessed 25 March 2019].
- [25] Singh, M.S., Pondenkandath, V., Zhou, B., Lukowicz, P. and Liwicki, M. (2017) Transforming Sensor Data to the Image Domain for Deep Learning - an Application to Footstep Detection.arXiv:1701.01077 [cs], Available from <http://arxiv.org/abs/1701.01077> [accessed 25 March 2019].